DATA SCIENCE

Don't Know Anything About ML? Start With These 5 Concepts

The 5 ideas every machine learning course is built on, explained with everyday examples and a few lines of code. Learn these and the rest makes sense.

INSIDE THIS PDF · 8 PAGES

- Concept 1: What a model is
- Concept 2: Supervised vs unsupervised
- Concept 3: Regression vs classification
- Concept 4: Train, test and overfitting
- Concept 5: How to judge a model
- Put the 5 together
- What to learn next

@thedataguy16

Concept 1: What a model is

A model is a rule learned from examples instead of written by hand.

- **Features**

  The inputs you know. For a house: area, city, number of rooms.

- **Target**

  The answer you want to predict. The house price.

- **Training**

  Show the model thousands of houses with known prices. It learns how features relate to price.

- **Prediction**

  Give it a new house without a price. It estimates one using what it learned.

Everyday example: an experienced broker guesses a flat's price from its area and locality. A model does the same, from data.

@thedataguy16

Concept 2: Supervised vs unsupervised

The first question for any ML problem: do you have the answers in your data?

- **Supervised learning**

  You have the target. Past loans labelled "repaid" or "defaulted". The model learns to predict it for new loans.

- **Unsupervised learning**

  No target. You have customer purchase data and want to find natural groups. The model finds the groups itself.

- **Most business ML is supervised**

  Churn, fraud, demand forecasting, lead scoring.

- **Common unsupervised uses**

  Customer segmentation, anomaly detection, recommendations.

@thedataguy16

Concept 3: Regression vs classification

Inside supervised learning, the type of target decides the type of model.

```
from sklearn.linear_model import (
    LinearRegression, LogisticRegression)

# regression: predict a number
LinearRegression().fit(X, sales)

# classification: predict a category
LogisticRegression().fit(X, churned)
```

- **Regression: predict a number**

  Next month's sales, delivery time in hours, house price.

- **Classification: predict a category**

  Will this customer churn: yes or no. Is this email spam. Which of 5 product categories.

- **Same data, different question**

  Predicting a customer's spend is regression. Predicting whether they will spend over ₹5,000 is classification.

@thedataguy16

Concept 4: Train, test and overfitting

A model that memorises is useless. You must test it on data it has never seen.

```
from sklearn.model_selection import
train_test_split
X_train, X_test, y_train, y_test =
train_test_split(
    X, y, test_size=0.2, random_state=42)
model.fit(X_train, y_train)
print(model.score(X_test, y_test))   # the
number that matters
```

Train set

Usually 80% of the data. The model learns from this.

Test set

The other 20%, hidden during training. This is the honest exam.

Overfitting

98% on train, 70% on test. The model memorised examples, like a student who learned past papers by heart.

Underfitting

Poor on both. The model is too simple to capture the pattern.

@thedataguy16

Concept 5: How to judge a model

A single "accuracy" number can lie. Pick the metric that matches the cost of mistakes.

- **Accuracy**

  Share of predictions that were right. Only fair when classes are balanced.

- **Precision**

  Of everything flagged as fraud, how much really was fraud. Matters when false alarms are costly.

- **Recall**

  Of all real fraud, how much we caught. Matters when missing a case is costly.

- **MAE for regression**

  Average error in real units. "Our forecast is off by ₹1,200 on average" is easy for anyone to understand.

Trap: if 1% of transactions are fraud, a model that always says "not fraud" is 99% accurate and catches nothing.

@thedataguy16

Put the 5 together

Every ML project follows this flow. You now know each step.

```
import pandas as pd
from sklearn.model_selection import (
    train_test_split)
from sklearn.ensemble import (
    RandomForestClassifier)
from sklearn.metrics import (
    classification_report)

df = pd.read_csv("customers.csv")
# 1. features and target
X = df[["tenure", "bill", "complaints"]]
y = df["churned"]
# 2 and 3. supervised classification
# 4. hold out a test set
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, stratify=y)
model = RandomForestClassifier()
model.fit(X_tr, y_tr)
# 5. judge on unseen data
pred = model.predict(X_te)
print(classification_report(y_te, pred))
```

@thedataguy16

What to learn next

With these 5 concepts in place, go in this order.

- **Week 1-2**

  Pandas for cleaning data. Most ML time is spent here.

- **Week 3-4**

  Linear and logistic regression, decision trees, random forest.

- **Week 5-6**

  Cross-validation, feature engineering, handling imbalanced data.

- **Week 7-8**

  One end-to-end project with a clear business question and a README.

Free practice data: Kaggle datasets on telecom churn, loan default and house prices fit these concepts well.

@thedataguy16